



Master 1 GIL — Technologies XML
**Projet : Service REST SEPA (ISO 20022) &
Déploiement Conteneurisé**

RAPPORT TECHNIQUE DE PROJET

Projet : SEPA26 — Service REST ISO 20022

Version : 1.0.0

Date : Octobre 2026

Auteurs : Umm-Habibah Ouattara & Florian Pépin

Technologies : Spring Boot 4.1.1, Java 25 LTS, Docker, MariaDB

Table des matières

1	Introduction	2
2	Contexte et Objectifs	2
3	Architecture Logicielle & Principes de Conception	3
3.1	Application des Principes SOLID	3
3.2	Application du Principe DRY (Don't Repeat Yourself)	3
3.3	Organisation du Dépôt & Arborescence	3
3.4	Stack Technique	4
4	Points d'Accès de l'Application	5
5	Usine Logicielle, Conteneurisation & CI/CD	5
5.1	Build Multi-Stage avec Docker	5
5.2	Orchestration Locale avec Docker Compose	5
5.3	Gestion des Variables d'Environnement	6
5.4	Pipeline d'Intégration Continue (GitHub Actions)	6
A	Annexe : Catalogue Exhaustif des Classes et Composants	7

1 Introduction

Ce projet a pour objectif la conception et l'implémentation d'un service web REST complet dédié à la gestion, la validation et la restitution de transactions de prélèvement bancaire SEPA, basées sur la norme internationale **ISO 20022** (pain.008.001.02).

Le service permet l'ingestion de messages XML, leur validation stricte selon un schéma XML Schema (XSD), leur stockage persistant dans une base relationnelle, leur recherche multicritères, ainsi que leur restitution sous forme de flux XML structurés ou de vues HTML conviviales transformées par feuilles de style XSLT 3.0.

L'application a été entièrement conçue selon les principes de **Clean Code** (**SOLID** et **DRY**) et s'accompagne d'une infrastructure moderne et autonome basée sur **Docker**, **Docker Compose** et un pipeline d'Intégration Continue (**GitHub Actions**).

2 Contexte et Objectifs

- **Cadre académique** : Master 1 Génie de l'Informatique Logicielle (GIL) — Université de Rouen Normandie.
- **Norme métier** : Format XML ISO 20022 pour les prélèvements SEPA directs (*Customer Direct Debit Initiation*).
- **Exigences de qualité logicielle** :
 - Séparation stricte des responsabilités (Single Responsibility Principle).
 - Inversion des dépendances et découplage via interfaces (Dependency Inversion Principle).
 - Élimination de la duplication de code (Don't Repeat Yourself).
 - Déploiement portable, reproductible et sans adhérence à une machine ou à un serveur externe.

3 Architecture Logicielle & Principes de Conception

L'architecture du projet repose sur un découpage en couches étanches, complété par des services spécialisés à responsabilité unique.

3.1 Application des Principes SOLID

- **Single Responsibility Principle (SRP)** : La logique XML n'est pas mélangée à la persistance ni aux contrôleurs.
 - `XmlValidationService` est l'unique responsable de la validation XSD.
 - `XmlSerializationService` est l'unique responsable de la conversion bidirectionnelle JAXB.
 - `XsltTransformationService` est dédié aux transformations XSLT vers HTML.
 - `EndpointDocumentationService` encapsule les métadonnées de documentation de l'API.
 - `SepaApiController` traite exclusivement les flux de données REST (XML), tandis que `SepaWebController` pilote l'interface utilisateur web (HTML Thymeleaf).
- **Open/Closed Principle (OCP) & Liskov Substitution (LSP)** : Tous les services clés sont définis à travers des interfaces Java. Les implémentations peuvent évoluer ou être substituées sans impacter les contrôleurs ou les consommateurs métier.
- **Interface Segregation Principle (ISP)** : Les interfaces de service sont granulaires et ciblées, évitant aux clients de dépendre de méthodes superflues.
- **Dependency Inversion Principle (DIP)** : L'injection par champ (`@Autowired` sur attributs privés) a été bannie au profit exclusif de **l'injection par constructeur** avec des attributs `final`. Cela garantit l'immuabilité des composants et facilite grandement l'isolation lors des tests unitaires sans nécessiter de contexte Spring complexe.

3.2 Application du Principe DRY (Don't Repeat Yourself)

- **Mise en cache des ressources lourdes** : L'instance de schéma XSD (`Schema`) et le contexte JAXB (`JAXBContext`) sont pré-instanciés une seule fois au démarrage de l'application, évitant de recharger et parser les schémas à chaque requête HTTP.
- **Templating modulaire Thymeleaf** : La barre de navigation et le pied de page sont factorisés dans un composant unique (`fragments/layout.html`) et réutilisés dans toutes les pages de l'application.
- **Centralisation CSS** : Tous les styles de l'application sont centralisés dans `sepa26.css` avec des variables CSS, évitant toute duplication de règles CSS inline.
- **Génération uniforme des erreurs** : Un formatage réutilisable des messages d'erreur évite l'éparpillement de chaînes XML codées en dur.

3.3 Organisation du Dépôt & Arborescence

Afin d'isoler nettement les outils d'infrastructure du code source applicatif, le dépôt est structuré comme suit :

- `app/` : Application Spring Boot autonome :
 - `src/main/java/` : Code source Java (contrôleurs, services, entités, DTOs).
 - `src/main/resources/` : Configuration, templates Thymeleaf, CSS et ressources XML/XSD/XSLT.
 - `src/test/` : Suites de tests unitaires et d'intégration.
 - `pom.xml` : Descripteur Maven de dépendances et de plugins.
 - `checkstyle.xml` : Règles de conformité et d'analyse statique du code (linter).
 - `Dockerfile` : Fichier de build multi-stage de l'image Docker.
- `doc/` : Documentation technique et sources LaTeX du présent rapport.
- `docker-compose.yml` : Orchestration locale des conteneurs applicatif et base de données.
- `.env` & `.env.example` : Paramètres de configuration et variables d'environnement.
- `.github/workflows/` : Pipeline d'Intégration Continue (GitHub Actions).

3.4 Stack Technique

- **Langage** : Java 25 (LTS).
- **Framework** : Spring Boot 4.1.1 (Spring Web, Spring Data JPA).
- **Gestion des dépendances & Build** : Apache Maven.
- **Qualité de code & Linter** : Maven Checkstyle Plugin 3.6.0.
- **Base de données** : MariaDB 11 en production / Docker, et H2 Database en mémoire pour l'exécution des tests.
- **Traitement XML** : Jakarta XML Binding (JAXB 4.0), Saxon-HE 12.9 (moteur XSLT 3.0), Jackson XML.
- **Moteur de rendu Web** : Thymeleaf.
- **Conteneurisation & CI** : Docker, Docker Compose, GitHub Actions.

4 Points d'Accès de l'Application

L'application expose ses fonctionnalités à travers deux vecteurs complémentaires : l'API REST ISO 20022 (`SepaApiController`) et l'IHM Web interactive (`SepaWebController`).

Méthode	Endpoint	Format	Description
GET	/	HTML	Page d'accueil du service
GET	/help	HTML	Documentation interactive des points d'accès
GET	/transfert	HTML	Formulaire d'importation et validation de flux XML
GET	/sepa26/resume/html	HTML	Tableau de bord des 10 dernières transactions
GET	/sepa26/resume/xml	XML	Synthèse XML des 10 derniers documents
GET	/sepa26/xml/{id}	XML	Fiche complète du document SEPA
GET	/sepa26/html/{id}	HTML	Rendu stylisé du document via feuille XSLT
POST	/sepa26/insert	XML	Insertion d'un flux SEPA (validation XSD)
DELETE	/sepa26/delete/{id}	XML	Suppression d'une transaction
GET	/sepa26/search	XML	Recherche multicritères (date, montant)

TABLE 1 – Synthèse des points d'accès de l'application SEPA26

5 Usine Logicielle, Conteneurisation & CI/CD

Pour garantir un déploiement fiable et reproductible sur n'importe quel environnement, le projet s'appuie sur une infrastructure conteneurisée complète.

5.1 Build Multi-Stage avec Docker

Le fichier `Dockerfile` adopte une approche multi-stage optimisant à la fois la sécurité et la légèreté de l'image finale :

1. **Étape de compilation (Builder)** : S'appuie sur l'image `eclipse-temurin:25-jdk-alpine`. Elle intègre Maven, compile le code source et génère le fichier `sepa26server.jar`.
2. **Étape d'exécution (Runtime)** : S'appuie sur l'image minimale `eclipse-temurin:25-jre-alpine`. Seul le fichier binaire produit y est copié. L'application est exécutée sous un utilisateur non-privilégié (`spring:spring`) pour une sécurité renforcée.

L'image produite est baptisée `sepa26-app:latest`.

5.2 Orchestration Locale avec Docker Compose

Le fichier `docker-compose.yml` orchestre les deux services nécessaires au fonctionnement complet du projet :

- **Service db** : Conteneur MariaDB 11 avec volume persistant (`mariadb_data`) et contrôle de santé (`healthcheck`).

- **Service app** : Conteneur applicatif `sepa26-app` exposé sur le port 8100, démarré automatiquement une fois la base de données saine et prête à recevoir des connexions.

Le lancement s'effectue en une commande unique :

```
docker compose up -build
```

5.3 Gestion des Variables d'Environnement

La configuration de l'infrastructure est externalisée via des fichiers d'environnement :

- `.env.example` : Modèle versionné décrivant les variables nécessaires (`SERVER_PORT`, `DB_URL`, `DB_USER`, `DB_PASSWORD`, `DB_ROOT_PASSWORD`).
- `.env` : Fichier de configuration local, ignoré par Git pour préserver la confidentialité des paramètres.

5.4 Pipeline d'Intégration Continue (GitHub Actions)

Un workflow GitHub Actions automatisé (`.github/workflows/ci.yml`) valide l'intégrité du code à chaque *push* et *pull request* :

1. **Vérification de la qualité du code (Checkstyle)** : Exécution du linter (`mvn checkstyle:check`) pour garantir l'absence d'imports non utilisés, le respect des conventions de nommage et la lisibilité du code selon les règles Clean Code.
2. **Compilation et tests (Build & Test)** : Mise en place de Java 25 (Temurin) avec cache Maven et exécution intégrale des suites de tests unitaires et d'intégration (`mvn clean test`).
3. **Validation du conteneur (Docker Build)** : Vérification automatisée de la construction de l'image Docker multi-stage à partir du contexte `app/` (`docker build -t sepa26-app:latest ./app`).

A Annexe : Catalogue Exhaustif des Classes et Composants

Ce catalogue détaille l'ensemble des classes Java développées au sein du projet SEPA26, avec leur localisation, leur rôle et les principes de conception associés.

Classe / Interface	Package	Rôle et Responsabilité
Sepa26Application	fr.univrouen.sepa26	Point d'entrée exécutable démarrant le contexte Spring Boot.
SepaApiController	...controllers	Contrôleur REST (@RestController) gérant tous les flux XML : consultation des résumés, détails, insertion validée, suppression et recherche.
SepaWebController	...controllers	Contrôleur Web (@Controller) gérant les vues HTML Thymeleaf : accueil, documentation, transfert, tableau récapitulatif et transformation XSLT.
SepaService	...services	Façade métier coordonnant la persistance, l'unicité des virements et les services de traitement XML.
XmlValidationService	...services	Interface du service de validation des flux XML contre les schémas XSD.
XmlValidationServiceImpl	...services.impl	Implémentation de la validation XSD avec mise en cache du schéma <code>sepa26.xsd</code> pour des performances optimales.
XmlSerializationService	...services	Interface du service de sérialisation et désérialisation XML JAXB.
XmlSerializationServiceImpl	...services.impl	Implémentation JAXB avec mise en cache du <code>JAXBContext</code> réutilisable.
XsltTransformationService	...services	Interface de transformation de documents XML en rendu HTML via XSLT.
XsltTransformer	...util	Implémentation du moteur Saxon-HE appliquant la feuille de style <code>sepa26.xslt</code> .
EndpointDocumentationService	...services	Interface fournissant les métadonnées de documentation des points d'accès.
EndpointDocumentationServiceImpl	...services.impl	Implémentation fournissant la liste descriptive des 10 endpoints pour la page d'aide.
DocumentRepository	...repository	Interface Spring Data JPA assurant les opérations de persistance, de recherche multicritère et de filtrage par <code>PmtId</code> .
DataInitializer	...init	Composant <code>CommandLineRunner</code> injectant deux documents SEPA types au démarrage si la base est vide.

Classe / Interface	Package	Rôle et Responsabilité
<code>Document</code>	<code>...model</code>	Entité racine JPA et JAXB représentant le document ISO 20022 complet.
<code>CstmrDrctDbtInitn</code>	<code>...model.Document</code>	Sous-entité encapsulant le bloc d'initiation du prélèvement.
<code>GrpHdr</code>	<code>...model.Document</code>	Entité d'en-tête de lot (identifiant de message, date, somme de contrôle, nombre d'opérations).
<code>PmtInf</code>	<code>...model.Document</code>	Entité d'information de paiement regroupant les informations créancier et les transactions.
<code>DrctDbtTxInf</code>	<code>...model.Document</code>	Entité de transaction élémentaire (débiteur, montant, mandat, compte).
<code>ValidationResult</code>	<code>...dto</code>	Record immutable contenant l'état de validation XSD et le message d'erreur éventuel.
<code>ParseResult<T></code>	<code>...dto</code>	Record générique encapsulant le résultat d'une désérialisation XML JAXB.
<code>SepaResponse</code>	<code>...dto</code>	DTO de réponse XML standardisé pour les opérations de modification (<code>INSERTED</code> , <code>DELETED</code> , <code>ERROR</code>).
<code>DocumentList</code>	<code>...dto</code>	DTO conteneur XML pour la sérialisation des listes de documents.
<code>SearchResponse</code>	<code>...dto</code>	DTO de réponse pour la recherche multicritères (<code>OK</code> , <code>NONE</code> , <code>ERROR</code>).
<code>LocalDateAdapter</code>	<code>...util</code>	Adaptateur JAXB assurant la conversion entre les dates ISO (<code>YYYY-MM-DD</code>) et <code>LocalDate</code> .
<code>LocalDateTimeAdapter</code>	<code>...util</code>	Adaptateur JAXB assurant la conversion des horodatages ISO (<code>YYYY-MM-DD'T'HH:mm:ss</code>) et <code>LocalDateTime</code> .